Accurate Self-Service Kubernetes Multi-Tenancy

Erik Godding Boye, Senior Consultant & Partner - Zenior

Cloud Native Oslo meetup, 14/10-2026





Who am I?

- Worked as Developer for almost 30 years
- The last years mainly working with platforms: Kubernetes, Kafka and Crossplane
- Currently working as a contractor on platforms for Statnett SF

- Passionate about open source and have contributed to a lot of open source projects
- cert-manager (and Accurate) Maintainer
- Newly appointed CNCF Ambassador

Multi-Tenancy

Multi-tenancy is the capability of a platform to serve multiple independently governed tenants through shared infrastructure.

While enforcing tenant-specific boundaries for:

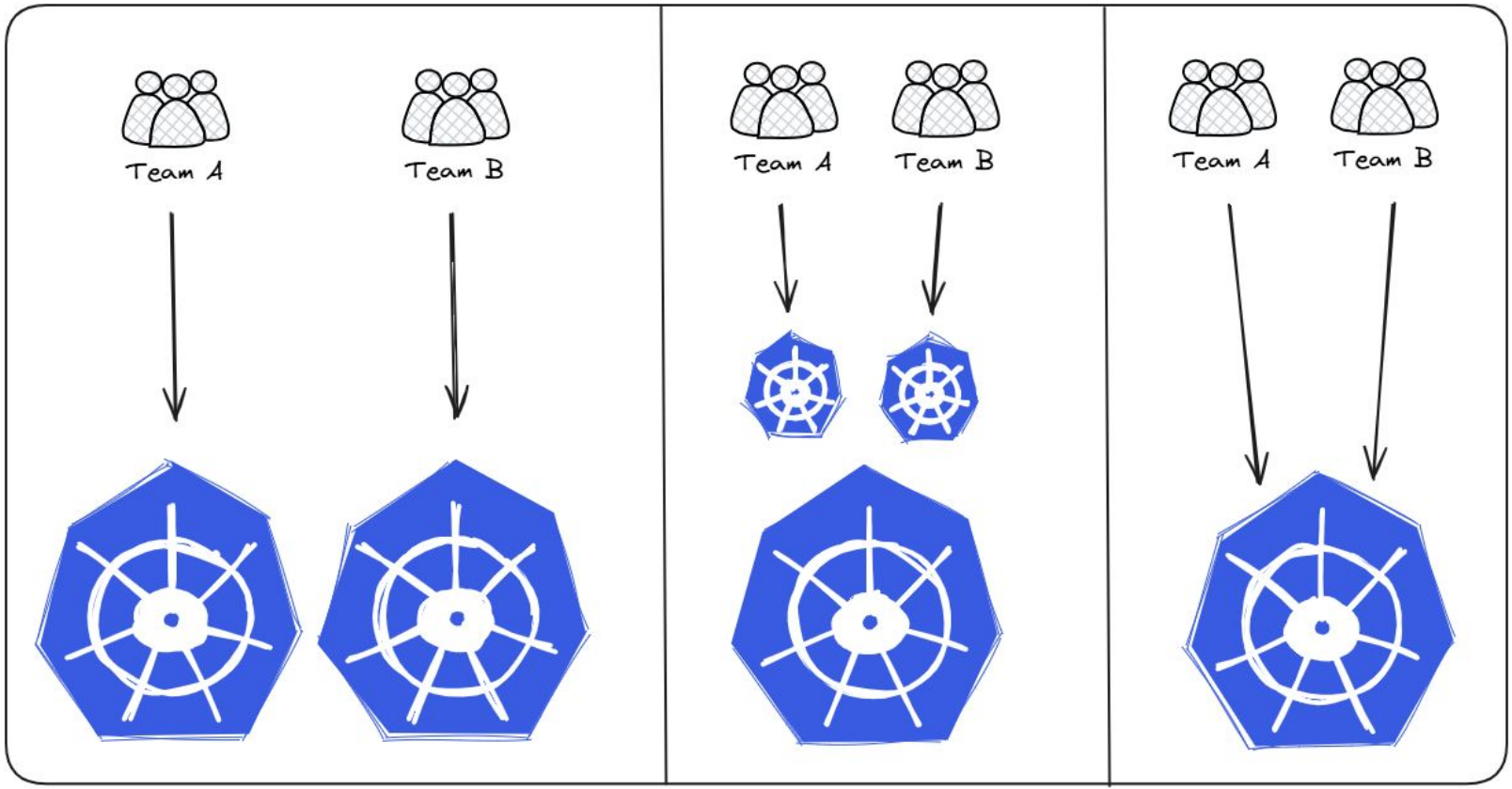
- Identity
- Authorization
- Resource consumption
- Workload isolation
- Lifecycle management

Kubernetes Multi-Tenancy models

Cluster-based tenancy: Each tenant receives a dedicated Kubernetes cluster, potentially sharing underlying cloud infrastructure.

Virtual-cluster tenancy: Tenants receive separate virtual Kubernetes control planes while sharing underlying infrastructure.

Namespace-based tenancy: Multiple tenants share a Kubernetes control plane and worker infrastructure, with namespace-based boundaries.



Kubernetes Multi-Tenancy models

Dimension	Namespace as a Service	Virtual Control Plane	Cluster as a Service
Kubernetes API	Shared	Dedicated per tenant	Dedicated per tenant
Control plane	Shared	Isolated	Isolated
Worker nodes	Shared	Shared by default	Dedicated to cluster, potentially cloud-shared underneath
Cluster-scoped resources	Centrally managed and shared	Tenant can manage its own virtual cluster resources	Tenant can manage cluster-scoped resources
Tenant autonomy	Limited	High	Highest, subject to platform restrictions
Isolation	Namespace and policy boundaries	Stronger control-plane isolation	Strong control-plane isolation; data-plane isolation depends on infrastructure
Infrastructure efficiency	Highest	High	Generally lowest
Operational overhead	Lowest	Medium	Highest
Best suited for	Teams and standard workloads	Tenants needing cluster-level autonomy	Tenants needing dedicated clusters or stronger isolation

Namespace as a Service

The cluster control plane resources like the API server and scheduler, and worker node resources like CPU, memory, etc. are available for use across all tenants.

To isolate tenant workloads, each namespace must also contain:

- role bindings: for controlling access to the namespace
- network policies: to prevent network traffic across tenants
- resource quotas: to limit usage and ensure fairness across tenants
- limit ranges: to set defaults and enforce constraints on requests/limits

Onboarding



1. Tenant requests a namespace
2. Platform admin authorizes the request OPTIONAL
3. Platform automation provisions it
    - Namespace with labels
    - Quotas, policies and defaults
    - RBAC
4. Tenant receives access and can deploy workloads

Tenant namespaces

apex

onyx

rift

Multiple namespace per tenant

A tenant might want multiple namespaces to create additional isolation and organizational boundaries within the tenant.

- Environment separation: dev, test, prod
- Operational ownership: managed by different teams within the tenant
- Temporary workloads: previews, experiments, or migrations
- Regulatory/data separation: separate workloads with different data-handling requirements
- +++



Accurate

- Accurate is a Kubernetes Operator for soft multi-tenancy environments.
- Currently maintained by Cybozu (and friends). To-become a CNCF project.
- Resembles the now archived kubernetes-sigs Hierarchical Namespace Controller (HNC).
- Features:
  - Tenant users can self-provision sub-namespaces using the SubNamespace custom resource.
  - Inheriting labels and annotations from parent namespaces
  - Propagate resources between namespaces
  - Template namespaces
  - kubectl plugin

Accurate onboarding

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: onyx
labels:
  accurate.cybozu.com/type: root
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: tenant-editors
  namespace: onyx
  annotations:
    accurate.cybozu.com/propagate: update
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: edit
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: team-onyx
```

SubNamespace custom resource

```yaml
apiVersion: accurate.cybozu.com/v2
kind: SubNamespace
metadata:
  name: onyx-acme
  namespace: onyx
```



```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: onyx-acme
```

Tenant namespace hierarchies



Namespace guardrails

- Namespace naming policies
  - root: ^(?P<root>.*)
    match: ^${root}-.*
- Circular references among namespaces are blocked by admission controls.
- Deleting a namespace that has one or more sub-namespaces is blocked by default.
  - Cascade delete can be enabled (admin opt-in).
- A SubNamespace can only be created in an Accurate namespace (root- or sub-namespace).
- Prevents namespace “takeover”
  - Any attempt will put SubNamespace in a conflicted state.

Sub-namespace metadata

```yaml
apiVersion: accurate.cybozu.com/v2
kind: SubNamespace
metadata:
  name: onyx-kafka
  namespace: onyx
spec:
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
  labels:
    app.kubernetes.io/component: kafka
    app.kubernetes.io/part-of: streaming-platform
```

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: onyx-kafka
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
  labels:
    app.kubernetes.io/component: kafka
    app.kubernetes.io/part-of: streaming-platform
```

Sub-namespace guardrails

1. labels/annotations from parent namespace
2. labels/annotations from SubNamespace **spec**

```
config:
  annotationKeys:
    - instrumentation.opentelemetry.io/*
  labelKeys:
    - app.kubernetes.io/*
  subNamespaceAnnotationKeys:
    - instrumentation.opentelemetry.io/*
  subNamespaceLabelKeys:
    - app.kubernetes.io/*
```

Resource propagation

- Accurate can propagate any namespace-scoped resource, including custom resources.
- Controlled by the *accurate.cybozu.com/propagate=<mode>* annotation:
  - *create*: the resource will be created in namespaces if missing.
  - *update*: the resource will be created/updated/deleted in namespaces.
- The admin controls which resources that can be propagated by:
  - Granting the Accurate controller RBAC to the resources
  - Enabling Accurate controller watches on the resources
- Use cases:
  - User RBAC (we have seen this already)
  - Image pull secrets
  - GitOps: ServiceAccount + RBAC

Accurate namespace types

Root: Namespace labeled with *accurate.cybozu.com/type=root*

Sub-namespace: Namespace labeled with *accurate.cybozu.com/parent=<name>*

Template: Namespace labeled with *accurate.cybozu.com/type=template*

Template namespaces

- Any namespace other than sub-namespaces can reference a template namespace with the *accurate.cybozu.com/template=<name>* label.
- Resource propagation works as in Accurate namespace hierarchies
- Useful to enforce (*mode=update*) or default (*mode=create*) resources in instance namespaces.
- Examples:
  - Default network policies like default-deny-ingress and default-deny-egress
  - ResourceQuota
  - LimitRange
  - EgressFirewall (proprietary OpenShift resource)

kubectl-accurate

- *kubectl-accurate* is a *kubectl* plugin for Accurate
- Features:
  - Hierarchical view of namespace trees
  - Show the information about a namespace
  - Operations for root-, template- and sub-namespaces
- Mainly an admin tool because of no end-user RBAC to namespaces
- Example (admin) use cases:
  - Convert a normal namespace to a sub-namespace under parent
  - Move sub-namespace under a new parent

kubectl-accurate examples

```
$ kubectl accurate template list
├── template1
├── template2
│   ├── reference1
│   └── reference2
└── template3
```

```
$ kubectl accurate list
├── root1
├── root2
│   └── sub1
├── root3
├── subroot1
│   └── sn1
└── subroot2
```

```
$ kubectl accurate ns describe root2
Name: root2
Type: root
# of children: 1

Resources:
Kind      Name      From      Mode
--------  --------  --------  --------
Role      role1     tmpl3     create
Secret    mysecret            create
```

Features

- Tenant:
  - Self-provision namespace hierarchies
  - Manage sub-namespace labels/annotations
  - Propagate resources in their namespace hierarchy
- Admin:
  - Strong governance with namespace naming and metadata policies
  - Template namespaces to manage resources in tenant namespace hierarchies
  - UI for operations through kubectl plugin

Gotchas

- RBAC may be removed before namespace cleanup completes
  - Race between GitOps reconciliation and Accurate finalization.
  - If RBAC is removed first, the GitOps resource may be unable to complete deletion.
- Resources are propagated as-is
  - Service account references with an explicit namespace remain unchanged.
  - References without a namespace resolve to the service account in the current namespace.

```
  kind: ClusterRole
  name: admin
subjects:
  - kind: ServiceAccount
    name: gitops-reconciler
    namespace: onyx
```



```
  kind: ClusterRole
  name: admin
subjects:
  - kind: ServiceAccount
    name: gitops-reconciler
```

Potential improvements

- Resource quota per tenant (namespace hierarchy)
  - Recognized use case, but hard to implement.
  - Maybe a standalone project or upstream Kubernetes support?
- Move sub-namespace without admin-involvement
  - There is a open PR to implement this.
  - A move is performed only when both sides explicitly express the same intent.
- Sub-namespace resource injection to solve finalization race with GitOps reconcilers
- Improve the kubectl plugin experience for tenants
- ????

Questions or comments?